

CHRIST (Deemed to be University)
Department of Computer Science
Master of Computer Applications

Course: MCA411-2 - PYTHON Programming

LAB Exercise I –

Demonstrate use of Python data structures, Tuple, Set, List, and Dictionary.

Date of assignment: 16-09-2026

Date of submission: 16-09-2026

Name : Aditya Kumar

Reg. No : 2647203

Class : 2 MCA – B

Domain : Clean Energy Systems & Grid Decarbonization

Lab 1

Lab Exercise I:

Demonstrate use of Python data structures, Tuple, Set, List, and Dictionary.

A. Create a tuple, a list, a set, and a dictionary containing records/items from your selected domain. Demonstrate how to add, update, delete, and access elements. Display the data appropriately.

a) Tuple:

Code:

```
# 1. TUPLE: Immutable Substation Site Metadata
print("=== 1. TUPLE DEMONSTRATION ===")
# Creation
site_meta = ("SITE_BLR_01", "Substation_North", 150.0, "Grid_Tied")
print("Initial Tuple:", site_meta)

# Accessing elements (using indexing & slicing)
print("Site ID (Index 0):", site_meta[0])
print("Substation Name (Index 1):", site_meta[1])
print("Max Capacity kW (Index 2):", site_meta[2])
print("Slice [1:3]:", site_meta[1:3])

# Tuples are immutable (we can not add/update/delete elements, for that we need to create new tuple)
# Adding Element: using tuple concatenation
updated_site_meta = site_meta + ("Active",)
print("Tuple after concatenation (Add):", updated_site_meta)

# Deleting tuple
del updated_site_meta
print("Tuple reference deleted successfully.\n")
```

Output:

```
=== 1. TUPLE DEMONSTRATION ===
Initial Tuple: ('SITE_BLR_01', 'Substation_North', 150.0, 'Grid_Tied')
Site ID (Index 0): SITE_BLR_01
Substation Name (Index 1): Substation_North
Max Capacity kW (Index 2): 150.0
Slice [1:3]: ('Substation_North', 150.0)
Tuple after concatenation (Add): ('SITE_BLR_01', 'Substation_North', 150.0, 'Grid_Tied', 'Active')
Tuple reference deleted successfully.
```

b) List:

Code:

```
# 2. LIST: Hourly Solar Generation Telemetry (kw)
print("=== 2. LIST DEMONSTRATION ===")
# Creation
solar_readings = [12.5, 24.0, 35.8, 48.2]
print("Initial List:", solar_readings)

# Accessing elements
print("First Reading (Index 0):", solar_readings[0])
print("Latest Reading (Index -1):", solar_readings[-1])

# Adding elements (using append and insert)
solar_readings.append(55.0)
solar_readings.insert(2, 29.5)
print("List after append(55.0) and insert(2, 29.5):", solar_readings)

# Updating element
solar_readings[1] = 26.4
print("List after updating index 1 to 26.4:", solar_readings)

# Deleting elements (using pop and remove)
removed_val = solar_readings.pop()      # removes last item
removed_ind = solar_readings.pop(1)     # removes vlaue at specific index
solar_readings.remove(29.5)             # removes specific value
print(f"List after pop() [removed {removed_val}], pop(1) [removed {removed_ind}] and remove(29.5):", solar_readings)
print()
```

Output:

```
=== 2. LIST DEMONSTRATION ===
Initial List: [12.5, 24.0, 35.8, 48.2]
First Reading (Index 0): 12.5
Latest Reading (Index -1): 48.2
List after append(55.0) and insert(2, 29.5): [12.5, 24.0, 29.5, 35.8, 48.2, 55.0]
List after updating index 1 to 26.4: [12.5, 26.4, 29.5, 35.8, 48.2, 55.0]
List after pop() [removed 55.0], pop(1) [removed 26.4] and remove(29.5): [12.5, 35.8, 48.2]
```

c) Set:

Code:

```
# 3. SET: System Alert and Diagnostic Event Codes
print("=== 3. SET DEMONSTRATION ===")
# Creation (sets eliminate duplicate entries)
diagnostic_codes = {"ERR_OVERTEMP", "STATUS_OK", "ERR_LOW_SOC", "STATUS_OK"}
print("Initial Set (duplicates eliminated):", diagnostic_codes)

# Accessing (using membershipoperator IN)
print("Is 'ERR_OVERTEMP' in set?:", "ERR_OVERTEMP" in diagnostic_codes)
print("Is 'ERR_VOLTAGE_SPIKE' in set?:", "ERR_VOLTAGE_SPIKE" in diagnostic_codes)

# Adding elements
diagnostic_codes.add("ERR_VOLTAGE_SPIKE")
diagnostic_codes.update(["WARN_HIGH_CURRENT", "STATUS_OK"])
print("Set after add() and update():", diagnostic_codes)

# Updating (Set union)
cleared_codes = {"STATUS_OK", "RESOLVED"}
diagnostic_codes = diagnostic_codes.union(cleared_codes)
print("Set after union with cleared codes:", diagnostic_codes)

# Deleting elements (remove and discard)
diagnostic_codes.remove("RESOLVED")      # raises error if not found
diagnostic_codes.discard("NON_EXISTENT") # safe discard
popped_code = diagnostic_codes.pop()
print(f"Set after remove('RESOLVED') and pop() [popped {popped_code}]:", diagnostic_codes)
print()
```

Output:

```
=== 3. SET DEMONSTRATION ===
Initial Set (duplicates eliminated): {'ERR_OVERTEMP', 'ERR_LOW_SOC', 'STATUS_OK'}
Is 'ERR_OVERTEMP' in set?: True
Is 'ERR_VOLTAGE_SPIKE' in set?: False
Set after add() and update(): {'ERR_VOLTAGE_SPIKE', 'ERR_OVERTEMP', 'STATUS_OK', 'ERR_LOW_SOC', 'WARN_HIGH_CURRENT'}
Set after union with cleared codes: {'ERR_OVERTEMP', 'STATUS_OK', 'ERR_VOLTAGE_SPIKE', 'ERR_LOW_SOC', 'WARN_HIGH_CURRENT', 'RESOLVED'}
Set after remove('RESOLVED') and pop() [popped ERR_OVERTEMP]: {'STATUS_OK', 'ERR_VOLTAGE_SPIKE', 'ERR_LOW_SOC', 'WARN_HIGH_CURRENT'}
```

d) Dictionary:

Code:

```
# 4. DICTIONARY: Battery Energy Storage System (BESS) Unit State
print("=== 4. DICTIONARY DEMONSTRATION ===")
# Creation
bess_unit = {
    "unit_id": "BESS_01",
    "capacity_kwh": 200.0,
    "soc_pct": 68.5,
    "cell_temp_c": 31.2,
    "state": "IDLE"
}
print("Initial Dictionary:", bess_unit)

# Accessing elements
print("Unit ID:", bess_unit["unit_id"])
print("Current SoC (%):", bess_unit.get("soc_pct"))

# Adding new key-value pair
bess_unit["voltage_v"] = 405.6
print("Dictionary after adding 'voltage_v':", bess_unit)

# Updating existing values
bess_unit["soc_pct"] = 72.0
bess_unit["state"] = "CHARGING"
print("Dictionary after updating 'soc_pct' and 'state':", bess_unit)

# Delete keys (pop and del)
removed_param = bess_unit.pop("voltage_v")
print(f"Dictionary after pop('voltage_v') [removed {removed_param}]:", bess_unit)
del bess_unit["capacity_kwh"]
print("Dictionary after del ['capacity_kwh']:", bess_unit)
```

Output:

```
=== 4. DICTIONARY DEMONSTRATION ===
Initial Dictionary: {'unit_id': 'BESS_01', 'capacity_kwh': 200.0, 'soc_pct': 68.5, 'cell_temp_c': 31.2, 'state': 'IDLE'}
Unit ID: BESS_01
Current SoC (%): 68.5
Dictionary after adding 'voltage_v':
{'unit_id': 'BESS_01', 'capacity_kwh': 200.0, 'soc_pct': 68.5, 'cell_temp_c': 31.2, 'state': 'IDLE', 'voltage_v': 405.6}
Dictionary after updating 'soc_pct' and 'state':
{'unit_id': 'BESS_01', 'capacity_kwh': 200.0, 'soc_pct': 72.0, 'cell_temp_c': 31.2, 'state': 'CHARGING', 'voltage_v': 405.6}
Dictionary after pop('voltage_v') [removed 405.6]:
{'unit_id': 'BESS_01', 'capacity_kwh': 200.0, 'soc_pct': 72.0, 'cell_temp_c': 31.2, 'state': 'CHARGING'}
Dictionary after del ['capacity_kwh']:
{'unit_id': 'BESS_01', 'soc_pct': 72.0, 'cell_temp_c': 31.2, 'state': 'CHARGING'}
```

B. Design and develop a menu-driven data management system for a complete data-processing solution using combinations of lists, tuples, sets, and Dictionaries.

The program should demonstrate Nested data structures, set operations, dictionary operations, List comprehension, set comprehension, Dictionary comprehension, sorting, searching, Aggregation, and data validation

Code:

```
'''
*** PART B: Menu-Driven Solar Microgrid & BESS Data Management System ***
'''

# A main dictionary storing site details.
'''
Each site contains:
- A tuple for fixed details (Location, Capacity kW)
- A list for recent power readings
- A set for unique alert codes
- Simple numeric values for battery percentage and temperature
'''
sites = {
    "SITE_1": {
        "info": ("Substation North", 150.0),      # Tuple
        "readings": [40.0, 55.0, 65.0],          # List
        "soc": 75.0,                              # Float (State of Charge %)
        "temp": 32.0,                             # Float (Temperature °C)
        "alerts": {"OK", "HIGH_TEMP"}             # Set
    },
    "SITE_2": {
        "info": ("Substation South", 90.0),
        "readings": [20.0, 30.0, 25.0],
        "soc": 25.0,
        "temp": 40.0,
        "alerts": {"LOW_BATTERY", "HIGH_TEMP"}
    },
    "SITE_3": {
        "info": ("Substation East", 200.0),
        "readings": [80.0, 95.0, 110.0],
        "soc": 85.0,
        "temp": 28.0,
        "alerts": {"OK"}
    }
}

def view_sites():
    print("\n--- Site Records ---")
    for s_id, d in sites.items():
        print(f"[{s_id}] {d['info'][0]} | Cap: {d['info'][1]} kW | SoC: {d['soc']}% | Temp: {d['temp']}°C")
        print(f"        Readings: {d['readings']} | Alerts: {d['alerts']}")

    # Comprehensions (list and dictionary)
    low_soc = [s_id for s_id, d in sites.items() if d["soc"] < 30.0]
    all_alerts = {a for d in sites.values() for a in d["alerts"]}
```

```

    avg_gen = {s_id: round(sum(d["readings"]) / len(d["readings"]), 2) for s_id, d
in sites.items()}

```

```

print("\nLow SoC (<30%):", low_soc)
print("All Alerts:", all_alerts)
print("Average Power (kW):", avg_gen)

```

```

def add_site():

```

```

    print("\n--- Add New Site ---")
    s_id = input("Enter Site ID: ").strip().upper()
    if s_id == "" or s_id in sites:
        print("Invalid or duplicate Site ID.")
        return

```

```

    loc = input("Enter Location: ").strip()
    if loc == "":
        print("Location cannot be empty.")
        return

```

```

    cap_str = input("Enter Capacity in kW (10 to 500): ").strip()
    if not (10 <= float(cap_str) <= 500):
        print("Invalid capacity value.")
        return

```

```

    soc_str = input("Enter Battery SoC % (0 to 100): ").strip()
    if not (0 <= float(soc_str) <= 100):
        print("Invalid SoC percentage.")
        return

```

```

    sites.update({
        s_id: {
            "info": (loc, float(cap_str)),
            "readings": [0.0],
            "soc": float(soc_str),
            "temp": 25.0,
            "alerts": {"OK"}
        }
    })
    print(f"{s_id} registered.")

```

```

def search_site():

```

```

    print("\n--- Search / Delete Site ---")
    s_id = input("Enter Site ID: ").strip().upper()

```

```

    if s_id in sites:
        d = sites[s_id]
        print(f"Found: {s_id} | {d['info'][0]} | Cap: {d['info'][1]} kW | SoC:
{d['soc']}%")
        ch = input("Delete this record? (y/n): ").strip().lower()
        if ch == "y":
            sites.pop(s_id)
            print(f"{s_id} removed.")
    else:
        print("Site not found.")

```

```

def compare_alerts():

```

```

print("\n--- Set Operations on Alerts ---")
id1 = input("Enter First Site ID: ").strip().upper()
id2 = input("Enter Second Site ID: ").strip().upper()

if id1 in sites and id2 in sites:
    a1 = sites[id1]["alerts"]
    a2 = sites[id2]["alerts"]
    print(f"{id1} Alerts: {a1}")
    print(f"{id2} Alerts: {a2}")
    print("Union:", a1 | a2)
    print("Intersection:", a1 & a2)
    print(f"Only in {id1}:", a1 - a2)
else:
    print("Invalid Site ID(s).")

def sort_sites():
    print("\n--- Sort Sites ---")
    print("1. By Capacity")
    print("2. By SoC")
    ch = input("Choice: ").strip()

    if ch == "1":
        res = sorted(sites.items(), key=lambda x: x[1]["info"][1], reverse=True)
        for s_id, d in res:
            print(f"{s_id}: {d['info'][1]} kW ({d['info'][0]})")
    elif ch == "2":
        res = sorted(sites.items(), key=lambda x: x[1]["soc"], reverse=True)
        for s_id, d in res:
            print(f"{s_id}: {d['soc']}%")
    else:
        print("Invalid choice.")

def show_aggregations():
    if not sites:
        print("No records available.")
        return

    print("\n--- Telemetry Aggregations ---")
    all_readings = [r for d in sites.values() for r in d["readings"]]
    total_cap = sum(d["info"][1] for d in sites.values())
    total_pow = sum(all_readings)

    print("Total Capacity (kW):", total_cap)
    print("Total Power Logged (kW):", total_pow)
    print("Max Power (kW):", max(all_readings))
    print("Min Power (kW):", min(all_readings))
    print(f"Average Power (kW): {total_pow / len(all_readings):.2f}")

# Main Menu
while True:
    print("\n--- MENU ---")
    print("1. View All Sites")
    print("2. Add Site")
    print("3. Search / Delete Site")
    print("4. Compare Alerts (Set Ops)")
    print("5. Sort Sites")

```

```

print("6. Show Aggregations")
print("7. Exit")

choice = input("Enter option (1-7): ").strip()
if choice == "1":
    view_sites()
elif choice == "2":
    add_site()
elif choice == "3":
    search_site()
elif choice == "4":
    compare_alerts()
elif choice == "5":
    sort_sites()
elif choice == "6":
    show_aggregations()
elif choice == "7":
    print("Exiting...")
    break
else:
    print("Invalid option.")

```

Output:

Main Menu:

```

--- MENU ---
1. View All Sites
2. Add Site
3. Search / Delete Site
4. Compare Alerts (Set Ops)
5. Sort Sites
6. Show Aggregations
7. Exit
Enter option (1-7): |

```

All Sites (using Comprehensions):

```

Enter option (1-7): 1

--- Site Records ---
[SITE_1] Substation North | Cap: 150.0 kW | SoC: 75.0% | Temp: 32.0°C
        Readings: [40.0, 55.0, 65.0] | Alerts: {'OK', 'HIGH_TEMP'}
[SITE_2] Substation South | Cap: 90.0 kW | SoC: 25.0% | Temp: 40.0°C
        Readings: [20.0, 30.0, 25.0] | Alerts: {'HIGH_TEMP', 'LOW_BATTERY'}
[SITE_3] Substation East | Cap: 200.0 kW | SoC: 85.0% | Temp: 28.0°C
        Readings: [80.0, 95.0, 110.0] | Alerts: {'OK'}

Low SoC (<30%): ['SITE_2']
All Alerts: {'OK', 'HIGH_TEMP', 'LOW_BATTERY'}
Average Power (kW): {'SITE_1': 53.33, 'SITE_2': 25.0, 'SITE_3': 95.0}

```

Add Site:

```

Enter option (1-7): 2

--- Add New Site ---
Enter Site ID: SITE_X
Enter Location: CHRIST
Enter Capacity in kW (10 to 500): 250
Enter Battery SoC % (0 to 100): 75
SITE_X registered.

```

Search:

```
Enter option (1-7): 3

--- Search / Delete Site ---
Enter Site ID: SITE_X
Found: SITE_X | CHRIST | Cap: 250.0 kW | SoC: 75.0%
Delete this record? (y/n): n
```

Search & Delete:

```
Enter option (1-7): 3

--- Search / Delete Site ---
Enter Site ID: SITE_X
Found: SITE_X | CHRIST | Cap: 250.0 kW | SoC: 75.0%
Delete this record? (y/n): y
SITE_X removed.
```

Compare Alert:

```
Enter option (1-7): 4

--- Set Operations on Alerts ---
Enter First Site ID: SITE_1
Enter Second Site ID: SITE_X
SITE_1 Alerts: {'OK', 'HIGH_TEMP'}
SITE_X Alerts: {'OK'}
Union: {'OK', 'HIGH_TEMP'}
Intersection: {'OK'}
Only in SITE_1: {'HIGH_TEMP'}
```

Sort:

```
Enter option (1-7): 5

--- Sort Sites ---
1. By Capacity
2. By SoC
Choice: 1
SITE_X: 250.0 kW (CHRIST)
SITE_3: 200.0 kW (Substation East)
SITE_1: 150.0 kW (Substation North)
SITE_2: 90.0 kW (Substation South)
```

Aggregation:

```
Enter option (1-7): 6

--- Telemetry Aggregations ---
Total Capacity (kW): 690.0
Total Power Logged (kW): 520.0
Max Power (kW): 110.0
Min Power (kW): 0.0
Average Power (kW): 52.00
```